Day 07

Vision, evals and observability

Build one thing, an **invoice photo → validated JSON** extractor. Then ask: does it work, how well, did my change break it, and what is it doing in production?

1 Vision LLMs · 2 Images & PDFs · 3 Tables & charts · 4 Pydantic extraction · 5 Schema vs rules · 6 Golden datasets · 7 Metrics & failures · 8 Regression tests · 9 Tracing & logging · 10 Monitoring · 11 Debugging

1 · models that can see

Vision LLMs: an image is just more tokens

invoice photo → cut into small **patches** → patches become **tokens** → read with your text, same message

```
{"role": "user", "content": [
  {"type": "input_text", "text": "What is the TOTAL?"},
  {"type": "input_image",
   "image_url": "...",
   "detail": "high"}          # low | high | auto
]}
```

Send a URL or base64. Formats: PNG, JPEG, WEBP, non-animated GIF.

same invoice, gpt-6-luna	answer	input tokens
`detail="low"`	2,867.55 ✓	264
`detail="high"`	2,867.55 ✓	1,267

Bigger image, more detail → **more input tokens**. Every model counts image tokens differently: check `usage.input_tokens`.

gpt-6-luna: $0.10 in / $0.50 out per 1M tokens → about **$0.0002 per invoice**.

2 · input types

Image and PDF inputs

Image

- `input_image`
- photos, screenshots, scans
- the model **looks**

Digital PDF

- `input_file` (base64 or file ID)
- has a **text layer**
- API extracts the text (+ page images)

Scanned PDF

- also `input_file`
- just a **picture inside a PDF**
- no text: the model must look

measured today	answer	input tokens
invoice_05.pdf (digital)	CloudNine Software LLC, $169.00 ✓	169
scanned_invoice.pdf (image only)	Fresh Basket Grocers, Rs. 2,867.55 ✓	6,027

rule of thumb: digital PDF at scale → pull the text yourself (cheapest). Layout matters, or it's a scan or photo → vision.

3 · document understanding

Tables and charts: the model reads, your code calculates

Table → "return the line items as a Markdown table"

```
| Description | Qty | Rate | Amount |
| A4 paper ream | 5 | 320.00 | 1,600.00 |
| Gel pens (box of 10) | 2 | 150.00 | 300.00 |
| Stapler | 1 | 245.00 | 245.00 |
| Sticky notes | 6 | 40.00 | 240.00 |
```

Copied exactly, alignment and all.

Chart → structured output: `ChartData(title, unit, bars)`

```
[('Apr', 4.2), ('May', 4.8), ('Jun', 3.9),
('Jul', 5.6), ('Aug', 6.1), ('Sep', 7.3)]
python: growth Apr → Sep = 73.8%
python: best month = Sep
```

All 6 bars read correctly.

why: turning pixels into numbers is the model's strength. Arithmetic "in its head" is where vision answers go wrong. **Extract → then compute in code.**

4 · the contract

Structured extraction with vision + Pydantic

```
# same as Day 05, now the input is an image
class Invoice(BaseModel):
    vendor_name: str
    invoice_number: str
    invoice_date: date        # a real date
    currency: str
    line_items: list[LineItem]
    subtotal: float
    tax: float
    total: float

r = client.responses.parse(
    model="gpt-6-luna", text_format=Invoice,
    instructions=PROMPTS["v1"],
    input=[{"role": "user", "content": [doc_part(path)]}],
    reasoning={"effort": "none"})
inv = r.output_parsed     # an Invoice object
```

- The Pydantic model is the **contract**: the API must return JSON that fits it
- "02 Oct 2026" came back as `2026-10-02`: the model normalises formats
- Prompt: *"Copy numbers exactly as printed. Do not calculate or correct anything."* We want what's **on the paper**. Checking it is our job.
- `reasoning: none`: extraction needs eyes, not thinking. Faster and cheaper.

5 · two kinds of checks

Schema validation vs business-rule validation

	schema validation	business-rule validation
asks	Is the **shape** right?	Is the **content** believable?
example	`total` is a number, date is a real date	qty × rate = amount, subtotal + tax = total, GST ∈ {0,5,12,18,28}
who	Pydantic + structured outputs (automatic)	your code: `check_rules(inv)`
on fail	retry the call, or reject	send to a human review queue

```
invoice_04, Gupta Caterers
subtotal 4600.0 + tax 230.0 = 4830.0
printed total = 4930.0
schema: OK | rules: ['total_mismatch', 'date_in_future']
```

total_mismatch: the **vendor** made a ₹100 mistake. Perfect JSON, wrong invoice.

date_in_future: the **model** read "30/09/2026" as the year 3009. Rules catch model mistakes too.

6 · the answer key

Golden datasets and test cases

"It worked on the one invoice I tried" is not a test. A **golden dataset** = real inputs + the **correct answer**, written down by a human.

file	what it tests	rule that should fire
`invoice_01.png`	clean, date 14/09/2026	-
`invoice_02.png`	different layout, date "02 Oct 2026"	-
`invoice_03.png`	bad phone photo: small, tilted, blurry	-
`invoice_04.png`	the vendor's total is wrong	total_mismatch
`invoice_05.pdf`	digital PDF, USD	-
`invoice_06.png`	8 line items, small font	-

Start: 20-50 real documents, hard cases on purpose.

Grow: every production failure becomes a new case.

Don't: tune the prompt on the same examples you score with.

7 · measure, then read the misses

Evaluation metrics and failure analysis

per-field accuracy, prompt v1, 6 invoices

vendor_name, currency, subtotal, tax, total, line_items	1.00
invoice_number	0.83
invoice_date	0.33

field accuracy **0.896** · doc exact match **0.333** · rules correct **0.833**

90% "accuracy" sounds fine. But only 2 of 6 invoices came back fully right.

every miss, then a cause

```
01: date expected 2026-09-14, got 1409-09-14
03: number expected MH-7731, got MH-773
03: date expected 2026-08-27, got 2026-02-27
04: date expected 2026-09-30, got 3009-02-20
06: date expected 2026-10-11, got 1110-11-10
```

day-first dates	4 of 5 misses → fix first
blurry photo	1 miss → better scan / review

metrics: field accuracy · doc exact match (straight-through rate) · rule-check accuracy · latency · cost per doc

8 · did my change break anything?

Regression testing for prompts and models

Rerun the golden set on every change. Compare with the saved baseline. **Any metric drops → don't ship.**

Change 1: prompt v2 · PASS

+ "Dates are day first (DD/MM/YYYY...): convert to YYYY-MM-DD."

field accuracy	0.896 → **0.958**
doc exact match	0.333 → **0.833**
rules correct	0.833 → **1.000**

Only the blurry photo still misses. v2 becomes the new baseline.

Change 2: gpt-4.1-nano · FAIL

Same prompt, "cheaper" model: same input price per token.

field accuracy	0.958 → **0.833**
doc exact match	0.833 → **0.167**
cost per doc	$0.000193 → **$0.000262**

Worse **and** dearer: it turns the image into more tokens.

in CI: a `pytest` that calls `regression_gate()`. Version prompts like code: `PROMPTS["v2"]`. Small sets are noisy: run each case 2-3 times.

9 · what happened, step by step

Observability: tracing and logging LLM calls

Evals test **your** examples. Observability shows what happens on **real traffic**. A **trace** = one request end to end. A **span** = one step inside it.



same `trace_id`, each span knows its `parent`. One JSON line per span → `logs/traces.jsonl`

Log on every LLM call

- trace_id, span name, parent
- model + **prompt version**
- input (file ID, **not** base64)
- output (parsed JSON)
- latency, tokens, cost
- status + error

never: API keys. Mask personal data.

Tools that do this with a UI: **Langfuse** (open source), **LangSmith**, **OpenTelemetry** GenAI conventions (`gen_ai.request.model`, `gen_ai.usage.input_tokens`).

10 · from logs to numbers

Monitoring failures and cost

A simulated day of traffic: 6 good invoices + 3 real-world failures, read back from the log with pandas.

traffic	result
6 golden invoices	5 auto_approve, 1 human_review
a chart uploaded by mistake	ValidationError (date 0000-01-01)
`max_output_tokens=60`	ValidationError (JSON cut off)
model `"gpt-6-lunaa"`	NotFoundError (404)

```
requests: 11 (9 here + 2 earlier demos)
error rate: 27%
LLM latency: P50 1858 ms | P95 2962 ms
total cost: $0.00158 ($0.000197 per call)

errors by type:
ValidationError 2
NotFoundError 1
```

alert when: error rate > 2% · P95 > 10 s · daily cost > budget · the share sent to human review jumps (model got worse, or a vendor changed their layout)

11 · "my upload failed"

Debugging a failed LLM request

find the **trace_id** → which **span** failed? → its **input** + **error** → fix + add to **golden set**

symptom in the trace	likely cause	fix
`NotFoundError 404, no tokens`	wrong model name in config	fix config, alert on deploy
`Invalid JSON: EOF while parsing`	output cut off by `max_output_tokens`	raise the limit, smaller schema
`ValidationError on one field`	not an invoice, or unreadable	tell the user; `is_invoice` check
`AuthenticationError 401`	bad or expired key	rotate the key, never retry
`429 / timeouts`	too much traffic	backoff + retry (Day 06)
`status ok, rule errors`	bad scan or bad extraction	open the image, human review
`status ok, rules ok, wrong`	silent failure	add it to the golden set

wrap up

Recap and homework

Remember

- Images and scans are **tokens**: measure the cost
- The model **reads**, your code **calculates**
- Schema = shape. Rules = content.
- No golden set, no idea if a change helped
- Log every call: you can't fix what you can't see

Homework

- Collect **10 real invoices or receipts** (remove personal data). Write their answers in `golden.json` format
- Run `run_eval`. Group every miss by cause
- Fix the biggest cause. Prove it with `regression_gate`

Sources, checked 2026-10-17: images and vision: developers.openai.com/api/docs/guides/images-vision · PDF input: developers.openai.com/api/docs/guides/pdf-files · prices: developers.openai.com/api/docs/models/gpt-6-luna, /gpt-4.1-nano · structured outputs: developers.openai.com/api/docs/guides/structured-outputs · Langfuse: langfuse.com/docs/observability/overview · LangSmith: docs.langchain.com/langsmith/observability · OpenTelemetry GenAI: github.com/open-telemetry/semantic-conventions-genai

All numbers from `day-07/code`, run 2026-10-17 on gpt-6-luna. Model answers vary run to run.